

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

1 Funkce - podprogram

Téma

1 Funkce - podprogram

Funkční volání

- Část podprogramu je volána opakovaně z různých míst programu:
 - ⇒ musí se chovat stejně jako skoky,
- po provedení podprogramu pokračuje hlavní program z místa, kde byla funkce volána:
 - ⇒ je nutné uchovávat návratovou hodnotu,
- může obsahovat parametry:
 - ⇒ předání pomocí určených registrů (A, X, ...),
 - ⇒ předání pomocí zásobníku (STACK),
- může mít návratovou hodnotu:
 - ⇒ téměř výhradně pomocí určeného registru (A, X, ...).

Instrukce CALL

Příklad: **CALL** tato_funkce ; skok na řádek tato_funkce

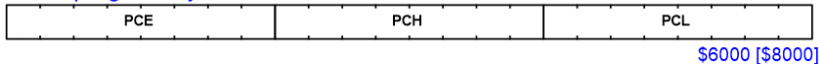
- Instrukce reprezentuje absolutní skok,
- **Program Counter** (programový čítač) je nejdříve zvýšen o délku instrukce **CALL** (viz dále),
- do zásobníku se uloží v pořadí hodnota **PCH** a **PCL** (spodní a prostřední byte **Program Counter**). Obdoba volání:

1 **PUSH** PCL,

2 **PUSH** PCH,

PC ... programový čítač

\$0000



- aktuální stav **PCH:PCL** je přepsán příslušnou adresou „tato_funkce“,

Délka a trvání instrukce CALL - programovací manuál

CALL

CALL Subroutine (Absolute)

CALL

Description The current PC register value is pushed onto the stack, then PC is loaded with the destination address in same section of memory. The CALL destination and the instruction following the CALL should be in the same section as PCE is not stacked. The corresponding RET instruction should be executed in the same section. This instruction should be used versus CALLR when developing a program.

dst	Asm	cy	lgth	Op-code(s)				ST7
longmem	CALL \$1000	4	3	CD	MS	LS		x
(X)	CALL(X)	4	1	FD				x
(shortoff,X)	CALL(\$10,X)	4	2	ED	XX			x
(longoff,X)	CALL(\$1000,X)	4	3	DD	MS	LS		x
(Y)	CALL(Y)	4	2	90	FD			x
(shortoff,Y)	CALL(\$10,Y)	4	3	90	ED	XX		x
(longoff,Y)	CALL(\$1000,Y)	4	4	90	DD	MS	LS	x
[shortptr.w]	CALL[\$10.w]	6	3	92	CD	XX		x
[longptr.w]	CALL[\$1000.w]	6	4	72	CD	MS	LS	
([shortptr.w],X)	CALL([\$10.w],X)	6	3	92	DD	XX		x
([longptr.w],X)	CALL([\$1000.w],X)	6	4	72	DD	MS	LS	
([shortptr.w],Y)	CALL([\$10.w],Y)	6	3	91	DD	XX		x

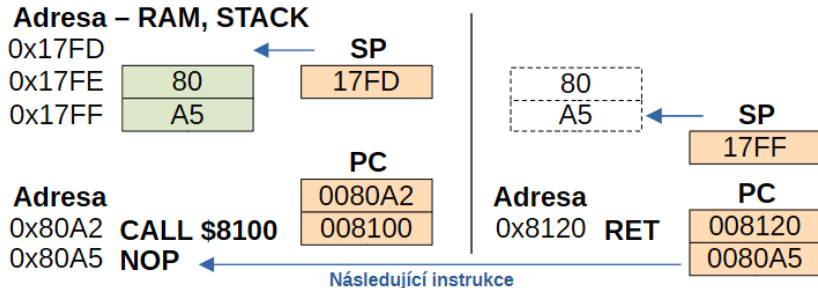
Instrukce RET

Příklad: RET ; skok na adresu načtenou ze zásobníku

- Instrukce reprezentuje absolutní skok - obnovuje původní stav **PCH:PCL**,
- ze zásobníku se získá v pořadí hodnota **PCH** a **PCL** a uloží se do Program Counter. Obdoba volání:
 - 1 **POP** PCH,
 - 2 **POP** PCL,
- **PCE** se nemění, tudíž cíl skoku i návratová adresa musí být ve stejném sekci adresy **PCE**,
- pro adresování celé paměti (delší skoky) lze použít kombinaci instrukcí **CALLF** a **RETF**, které upravují celou hodnotu **PCE:PCH:PCL**,
- varianta s relativním adresováním je s instrukcí **CALLR** - spoří místo v paměti (2 byte),

Funkce a zásobník

Návratová adresa je uložena v zásobníku:



K datům v zásobníku lze přistupovat indexovým adresováním. Toho lze využít několika způsoby:

- pomocí zásobníku můžeme předávat parametry funkce,
- v zásobníku můžeme vytvářet lokální proměnné dané funkce,
- můžeme měnit návratovou adresu funkce.

Jazyk C, překladač pro STM8 COSMIC

Volání funkce:

- Název funkce v **C** je na začátku překladačem doplněn o znak „_“ (krátká adresa - **CALL**) nebo o „f_“ (dlouhá adresa - **CALLF**),
- pokud má funkce jeden parametr, tak je předán pomocí reistru **A** (BYTE) nebo **X** (WORD) a po skoku instrukcí **CALL** je uložen do zásobníku,
- další parametry se ukládají do zásobníku před voláním instrukce **CALL**

Příklad: (char = byte)

char tatoFunkce(**char** arg1, **char** arg2, **char** arg3)

STACK:

SP	SP+1	SP+2	SP+3	SP+4	SP+5
—	arg1	PCH	PCL	arg2	arg3

Jazyk C, překladač pro STM8 COSMIC

Lokální proměnné

- vytvářejí se zásadně v zásobníku, proměnná má platnost jen po dobu trvání funkce,
- funkce může volat sama sebe - **REKURZE**.

Příklad: `char tatoFunkce(void) {`

```

    char    promenna0 = 0;
    char    promenna1 = 0;
    char    promenna1 = 0;
    return  0; }

```

STACK:

SP	SP+1	SP+2	SP+3	SP+4	SP+5
—	promenna2	promenna1	promenna0	PCH	PCL

Jazyk C, překladač pro STM8 **COSMIC**

Návratová hodnota

- u většiny překladačů (podle typu procesoru) se používá některý ze systémových registrů,
- registr **A** je pro návratovou hodnotu velikosti 1 BYTE nebo stejně velký ukazatel,
- registr **X** je pro návratovou hodnotu velikosti 1 WORD nebo stejně velký ukazatel,
- pro více bytové návratové hodnoty je vyhrazeno speciální paměťové místo v RAM (**c_lreg**).

Následující přednáška

- Volání funkcí a STACK,
- přerušení.

Děkuji za pozornost